

Meso2D Refactor EN Symbol Reference

Contents

Meso2D_refactor_EN Program Symbol Reference

Scope

This document lists the classes, methods, and variables used in the main application file Meso2D_refactor_EN.py. The focus is on each symbol's functionality inside the program flow.

Program Classes

Class	Purpose in the Program
Selector	Initial mode selection window. Lets the user choose circles, ellipses, or polygons before opening the main window.
About	Informational label/dialog with project meta-data and contact information.
mainProgram	Main application window. Handles UI events, project I/O, simulation setup, progress logging, and structure export.
Viewer_image	Secondary dialog used to visualize and save the generated structure image.
Controlador	Startup coordinator. Creates the selector window and opens the main window after mode selection.

External Services and Classes Used

Symbol	Source	Functionality in this Program
Ui_MainWindow	src.ui.Main03_EN	Generated Qt UI class for the main application window.
Menuinicio (Ui_MainWindow alias)	src.ui.Main_inicio_EN	Generated Qt UI class for the mode selector window.
Ui_Dialog_GV	src.ui.dialog_GV_EN	Generated Qt UI class for the image viewer dialog.
MiGraphicsView	src.ui.GV_EN	Custom graphics view widget used to display generated images with viewer controls.

SimulationController	src.simulation.orchestration_EN	Starts and stops the appropriate simulation worker based on selected mode.
build_project_payload	src.io.project_EN	Collects current in-memory state into a project dictionary for persistence.
save_project_to_file	src.io.project_EN	Writes serialized project payload to disk.
load_project_from_file	src.io.project_EN	Reads and parses project file from disk.
apply_project_payload	src.io.project_EN	Applies loaded payload fields to the active mainProgram instance.
export_structure_file	src.io.structure_export_EN	Performs format-aware structure export for DXF/SVG/JSON/GEO.
StructureExportError	src.io.structure_export_EN	Domain exception used to report export validation/runtime issues.
save_current_view	src.visualization.service_EN	Saves current rendered structure image from the main window context.
save_viewport_image	src.visualization.service_EN	Saves rendered image from the viewer dialog context.

Methods by Class

Selector

Method	Functionality
__init__	Builds selector UI and connects mode buttons to their handlers.
pb_circulos_pulsar	Emits mode "circulos" and closes selector.
pb_ellipses_pulsar	Emits mode "ellipses" and closes selector.
pb_poligonos_pulsar	Emits mode "poligonos" and closes selector.

--	--

About

Method	Functionality
<code>__init__</code>	Creates the informational text label and centers text alignment.
<code>initUI</code>	Delegates window-centering behavior.
<code>center</code>	Moves the widget to screen center based on available geometry.

mainProgram

Method	Functionality
<code>__init__(mode)</code>	Initializes main UI, connects menu/actions and buttons, sets default state, and creates SimulationController.
<code>sobre_programa</code>	Opens About window.
<code>close_application</code>	Asks for confirmation and exits the app if accepted.
<code>llenar_data_tabla_dosif</code>	Fills the gradation table from sieve_size/tpp lists; clears if data is missing.
<code>add_fila</code>	Adds one row to gradation table.
<code>open_project</code>	Handles "open project" workflow with overwrite confirmation when data already exists.
<code>open</code>	Reads project file from disk and loads values into UI/state.
<code>variables_limpias</code>	Resets internal state variables to start from a clean project context.
<code>new_project</code>	Starts a new project with optional cleanup confirmation.
<code>load_example</code>	Loads an example project, supporting legacy Spanish filenames.

llenar_data	Reflects internal state values into UI controls and checkboxes.
update_data(diccionario)	Maps legacy key aliases and applies payload values to current object.
save_project	Serializes and writes project data to a text file.
plotear(image)	Displays generated image in viewer, updates UI state, and logs simulation completion/-time.
run	Main entry from Run button: validates replacement of previous structure, updates data, and triggers simulation checks.
comprobar_data_para_worker	Verifies required simulation inputs; only starts simulation when all required values are present.
simulate	Sets random seed and feature toggles, updates UI progress controls, and delegates execution to SimulationController.start().
stop	Compatibility wrapper for stopping current simulation controller.
stop_simulation	Stops current simulation controller (same behavior as stop).
limpiar	Clears generated structure and temporary lists to prepare for another simulation run.
update	Pulls values from UI fields into numeric state variables and rebuilds derived areas and gradation arrays.
habilitar	Enables/disables related UI fields based on active checkboxes (pores, points, seed, etc.).
ver_structure	Opens image viewer dialog.
pasar_list_pores(list)	Receives pore list emitted by worker/controller.
pasar_list_coarse(list)	Receives coarse aggregate list emitted by worker/controller.

<code>pasar_list_fine(list)</code>	Receives fine aggregate list emitted by worker/controller.
<code>pasar_list_reactive(list)</code>	Receives reactive point list emitted by worker/controller.
<code>exportar_structure</code>	Generic export entrypoint. Exports to DXF/SVG/JSON/GEO through <code>structure_export_EN</code> service.
<code>exportar_structure_circulos</code>	Legacy/manual DXF export routine for circle-based structure representation.
<code>exportar_structure_ellipses</code>	Legacy/manual DXF export routine for ellipse-based structures (polyline approximation).
<code>points_ellipse</code> (nested helper)	Generates sampled points of a rotated ellipse for DXF polyline export.
<code>exportar_structure_poligonos</code>	Legacy/manual DXF export routine for polygonal structures, handling Shapely or vertex-list inputs.
<code>extract_polygon_vertices</code> (nested helper)	Normalizes polygon sources to a clean vertex list for DXF writing.
<code>save_image</code>	Saves current rendered structure image using visualization service.
<code>informar2</code>	Replaces trailing placeholder line (...) in info log or appends a new colored message.
<code>log</code>	Appends a colored message to information panel.
<code>log_worker</code>	Appends worker informational message in standard text color.
<code>log_worker_error</code>	Appends worker error message in error color.
<code>show_progress(percentage)</code>	Updates progress bar value from worker/controller progress signal.

Viewer_image

Method	Functionality
<code>__init__</code>	Builds dialog UI, creates graphics viewer widget, and wires save/close buttons.
<code>save</code>	Saves viewport image through visualization service.
<code>close</code>	Hides the image dialog window.

Controlador

Method	Functionality
<code>__init__</code>	Creates selector and binds selected mode signal to main window creation.
<code>open_mainprogram(mode)</code>	Creates and shows mainProgram for selected simulation mode.

Variables and Attributes

Module-level Variables

Variable	Functionality
<code>app</code>	QApplication instance, required to run Qt event loop.
<code>c</code>	Controlador instance that bootstraps the UI flow.

Selector Attributes

Attribute	Functionality
<code>mode_escogido</code>	Qt signal carrying selected mode string to Controlador.

mainProgram Attributes

Configuration and Input Data

Attribute	Functionality
mode	Current simulation mode: círculos, ellipses, or polígonos.
sieve_size	Sieve diameter list used for gradation fractions.
tpp	Cumulative passing percentage list aligned with sieve_size.
x, y	Specimen dimensions.
Pagg	Aggregate area fraction coefficient.
Pporos	Pore area fraction coefficient.
dporo_min, dporo_max	Minimum and maximum pore diameters.
r_react	Reactive radius parameter (currently not used directly in this file).
dpto_max_aggregates, dpto_min_aggregates	Reactive point diameter limits for aggregates.
dpto_max_pasta, dpto_min_pasta	Reactive point diameter limits for paste matrix.
Ppto_react_aggregates	Reactive point area fraction in aggregates.
Ppto_react_pasta	Reactive point area fraction in paste.
seed	Random seed used for reproducibility.
data	Canonical project key list/payload tracking used for load/save.
js	Loaded project dictionary from disk.
project	Payload dictionary generated for saving.
data_necesarios	Runtime list used to validate that required values are present before simulation.

Derived Numerical State

Attribute	Functionality
A	Total specimen area ($x*y$).
Aagg	Area allocation per aggregate gradation interval.
A_remanente	Remaining unassigned area after particle generation.
A_pores	Target total pore area.
A_points_aggregates	Target total reactive-point area over aggregates.
A_points_pasta	Target total reactive-point area in paste.
num_particulas	Counter used in particle generation workflows.
particulas	Aggregate/particle count bookkeeping by fraction.
radios	Generic radius list used by worker workflows.
radios_pores	Pore radius list pending placement.
radios_points	Global reactive-point radii trace.
radios_points_aggregates	Reactive-point radii assigned to aggregate placement.
radios_points_pasta	Reactive-point radii assigned to paste placement.

Geometry Collections and Export Data

Attribute	Functionality
list_aggregates	Runtime aggregate geometry list.
todos_aggregates	Render-ready aggregate patch/object list.
list_aggregates_coarse	Coarse aggregate export/render list.
list_aggregates_fine	Fine aggregate export/render list.

list_pores	Pore geometry list used for render/export.
todos_pores	Render-ready pore patch list.
list_ptos_react	Combined reactive-point list for export/render.
list_ptos_react_aggregates	Reactive-point list located on aggregates.
list_ptos_react_pasta	Reactive-point list located in paste.
todos_ptos_react	Render-ready reactive-point patch list.

UI and Execution Control State

Attribute	Functionality
dlg	Viewer_image dialog instance.
pop	About dialog/label instance.
simulation_controller	Orchestration service that selects/starts/stops the worker by mode.
start_time	Simulation start timestamp used for elapsed-time logging.
existe_structure	Indicates whether a generated structure already exists in current session.
todo_correcto	Indicates whether mandatory data validation passed.
check_pores	Flag to generate pores in current run.
check_points	Flag to generate reactive points in current run.
check_points_aggregates	Flag to generate reactive points on aggregates.
check_points_pasta	Flag to generate reactive points in paste.
aggregates_puestos	Flag indicating aggregates were already placed.
pores_puestos	Flag indicating pores were already placed.

redColor, blackColor, blueColor	Text colors used for log severity/status visualization.
---------------------------------	---

Viewer_image Attributes

Attribute	Functionality
gv_viewer	Graphics view widget used to display the rendered structure image.

Controlador Attributes

Attribute	Functionality
selector	Selector window instance for initial mode selection.
mainprogram	Main window instance created after mode selection.

UI Attributes Referenced from Ui_MainWindow

These widgets are not created manually in this file, but are heavily used by methods above.

Attribute	Functionality
tabla_dosif	Gradation table (sieve_size, tpp).
pb_run, pb_stop	Start/stop simulation buttons.
pb_add, pb_remove	Add/remove-update row interactions.
pb_ver_structure	Opens rendered structure viewer.
cb_pores, cb_points, cb_points_aggregates, cb_points_pasta, cb_seed	Feature toggle checkboxes that control simulation options and field enabling.
le_x, le_y, le_seed	Input fields for dimensions and random seed.
le_dmin, le_dmax, le_Pporos	Pore parameter input fields.
le_dmin_ptos_aggregates, le_dmax_ptos_aggregates, le_Pptos_aggregates	Reactive-point-on-aggregates input fields.

le_dmin_ptos_pasta, le_dmax_ptos_pasta, le_Pptos_pasta	Reactive-point-in-paste input fields.
dsb_coef_f	Numeric control for aggregate area fraction.
progressBar	Simulation progress indicator.
tb_info	Text log panel for status/error/progress messages.
a_Open, a_GuardarP, a_New, a_CargarE, a_Structure, a_GuardarI, a_Exit, a_About	Menu actions for project and application life-cycle.

Execution Flow Summary

1. Controlador shows Selector.
2. Selector emits chosen mode.
3. Controlador opens mainProgram(mode).
4. mainProgram gathers inputs, validates required values, and starts SimulationController.
5. Worker emits geometry/image/progress messages back to mainProgram.
6. mainProgram allows visualization and export once structure data is available.

Worker Methodology and Algorithms

This section describes the numerical and geometric methodology used by workers for aggregate and reactive-point generation/placement.

Worker Families and Morphology

Worker	Morphology and Data Representation
WorkerTodos (src/workers/- Worker_clusters_EN.py)	Circular aggregates and pores represented as [x, y, r]. Collision checks are mainly vectorized distance checks.
WorkerElipses (src/workers/- Worker_ellipses_EN.py)	Elliptical aggregates represented by [x, y, a, b, angle], converted to Shapely geometry for robust placement collision checks.
WorkerPoligonos (src/workers/- Worker_poligonos_EN.py)	Irregular polygonal aggregates represented as Shapely Polygon objects; pores can be circles or polygons depending on stage/export path.

1) Gradation Preprocessing and Area Budgeting

All workers follow the same base strategy:

1. Remove ultra-fine classes: gradation_sin_extrafinos keeps sieve sizes ≥ 2 mm.
2. Compute specimen area: $A = x * y$.
3. Split aggregate area per gradation interval using:

```
fraccion_i = (TPP_i - TPP_
i+1
) / (TPP_0 - TPP_
end
)
Aagg_i = fraccion_i * Pagg * A
```

1. Classify each interval into:
 - Coarse fractions: sieve > 4 mm
 - Fine fractions: sieve ≤ 4 mm
1. Keep $A_{\text{remanente}}$ between intervals so area not consumed in one step is transferred to the next one.

Functional objective:

- Preserve global aggregate proportion P_{agg} while respecting the gradation curve distribution.

2) Aggregate Size/Shape Sampling by Worker

WorkerTodos (Circular)

- Coarse and fine particles are sampled by random diameter within each sieve interval.
- Particle area is circle area $\pi * (d/2)^2$.
- New particles are accumulated while interval target area is not exceeded.
- Output is a radius list for later spatial placement.

WorkerElipses (Elliptical)

- Diameter d is sampled from sieve interval.
- Minor semi-axis $b = d/2$.
- Aspect ratio is sampled uniformly in $[1.0, 1.5]$, then $a = b * \text{aspect}$.
- Ellipse area $\pi * a * b$ is consumed against interval area budget.
- Placement later uses real Shapely ellipses generated from a unit circle scaled and rotated.

WorkerPoligonos (Polygonal)

- For each candidate particle, target area is derived from sampled diameter equivalent area.
- `generar_poligono_irregular` creates irregular polygons by:
- Sampling number of sides (5..10)
- Perturbing radial distances
- Scaling vertices to match target area
- Accepting if relative area error < 5
- Invalid geometries are cleaned using `buffer(0)`.

3) Spatial Placement of Aggregates

All workers use stochastic placement with geometric constraints:

1. Sample random position (and random angle for anisotropic shapes).
2. Enforce domain containment (full shape inside specimen).
3. Reject overlap with already placed aggregates.
4. Reject overlap with already placed pores (if pores were already generated).
5. Repeat up to max attempts (typically 3000) to avoid infinite loops in dense domains.

Differences by worker:

- **WorkerTodos:**
- Distance-only checks with NumPy arrays for circles.
- Fast and simple, optimized for `[x, y, r]` datasets.
- **WorkerElipses:**
- Shapely intersects/contains checks on true ellipse geometries.
- More robust for anisotropic shapes and rotations.
- **WorkerPoligonos:**
- Shapely polygon intersection/containment checks.
- Includes geometry cleaning and explicit placement loop bounds.

4) Pore Computation and Placement

Pore methodology is also budget-based:

1. Target pore area = $A * P_{poros}$.
2. Sample pore radius in $[d_{poro_min}/2, d_{poro_max}/2]$.
3. Subtract each pore area from remaining target until threshold.
4. Place pores randomly while enforcing:
 - Full containment in specimen
 - No overlap with aggregates
 - No overlap with previously placed pores

WorkerPoligonos and WorkerElipses use geometry-aware checks when aggregate geometry is non-circular.

5) Reactive Point Computation

Reactive points are generated in two channels:

- On aggregates ($P_{pto_react_aggregates}$, $d_{pto_min_aggregates}$, $d_{pto_max_aggregates}$)
- In paste ($P_{pto_react_pasta}$, $d_{pto_min_pasta}$, $d_{pto_max_pasta}$)

Both channels use area budgeting:

1. Target reactive-point area = $A * ratio$.
2. Sample radius in diameter range.
3. Subtract circle area $\pi * r^2$ until budget is consumed.
4. Keep separate radius lists for aggregate channel and paste channel.

6) Reactive Point Placement: Clusters and Noise

Workers implement two distinct spatial generators for reactive points:

A) Cluster-based placement (typically for points on aggregates)

- Generate cluster centers in the domain.
- Number of clusters is adapted from domain size using influence area:
 - $radio_influencia = 3 * sigma$
 - $area_cluster = \pi * radio_influencia^2$
 - $num_clusters = \max(k_minimo, \text{ceil}(\text{domain_area} / \text{area_cluster}))$
- Candidate points are sampled from normal distributions around random centers:

$$x \sim N(cx, sigma), y \sim N(cy, sigma)$$

- Candidates must satisfy placement constraints:
- Inside domain
- No overlap with pores and existing points
- Intersect aggregates when channel is "on aggregates"

Effect:

- Creates spatial grouping and localized concentration of reactive points.

B) Simplex-noise placement (typically for points in paste)

- Uses OpenSimplex noise field with fixed seed (42 in current code).
- Candidate point is accepted by noise gate:

$\text{valor} = (\text{noise2}(\text{x} \cdot \text{escala}, \text{y} \cdot \text{escala}) + 1) / 2$
 accept if $\text{valor} \geq \text{umbral}$

- Then standard geometric constraints are checked.

Main controls:

- *escala*: spatial frequency of noise map.
- *umbral*: selectivity (higher threshold -> fewer accepted zones).

Effect:

- Produces non-uniform, organic spatial patterns without sharp grid artifacts.

7) Collision Logic Summary

Context	Acceptance Rule
New aggregate	Must be fully inside domain and non-overlapping with existing aggregates (and pores if present).
New pore	Must be inside domain and non-overlapping with aggregates and pores.
New reactive point on aggregate	Must intersect at least one aggregate and avoid forbidden overlaps.
New reactive point in paste	Must avoid aggregate overlap (except intended logic per worker), pass noise/cluster sampling gate, and avoid forbidden overlaps.

8) Numerical/Implementation Safeguards

- Explicit max attempt loops to avoid deadlocks in saturated domains.
- Geometry repair (buffer(0)) for invalid polygons/derived shapes.
- Progress signaling by pipeline stage.
- Stop checks (check_stop) interleaved in loops for responsive cancellation.
- Emission ordering in geometry-based workers: data lists first, image later, to avoid stale export payloads.

9) Worker Pipeline (Common Pattern)

1. Gradation cleanup.
2. Area split by fraction.
3. Coarse aggregate generation by area.
4. Fine aggregate generation by area.
5. Aggregate placement in domain.
6. Optional pore generation and placement.
7. Optional reactive-point generation and placement (clusters/noise as configured).
8. Render/emit geometry and image to UI.

Pseudocode by Stage

This pseudocode summarizes the implementation logic in a code-maintenance-friendly format.

A) Common Simulation Pipeline

```
function simulate_worker(params, flags):
    assert required inputs are valid

    gradation = remove_ultra_fines(params.sieve_size, threshold=2.0)
    area_budget = split_area_by_gradation(gradation, params.tpp, params.Pagg,
    params.x * params.y)

    coarse_budget, fine_budget = classify_by_sieve(area_budget, split_mm=4.0)

    coarse_shapes = generate_coarse_aggregates(coarse_budget)
    fine_shapes = generate_fine_aggregates(fine_budget)

    placed_coarse, placed_fine = place_aggregates(coarse_shapes, fine_shapes,
    domain)

    if flags.check_pores:
        pore_radii = generate_pore_radii(params.Pporos, params.dporo_min,
        params.dporo_max)
        placed_pores = place_pores(pore_radii, domain, placed_coarse +
        placed_fine)
```

```

        if flags.check_points:
            r_agg = generate_reactive_radii(params.Ppto_react_aggregates, params.
dpto_min_aggregates, params.dpto_max_aggregates)
            r_paste = generate_reactive_radii(params.Ppto_react_pasta, params.
dpto_min_pasta, params.dpto_max_pasta)

            placed_points_agg = place_points_on_aggregates_clustered(r_agg, domain
, aggregates, pores)
            placed_points_paste = place_points_in_paste_simplex(r_paste, domain,
aggregates, pores)

            emit geometry lists first
            emit rendered image second

```

B) Gradation and Area Split

```

function split_area_by_gradation(sieve, tpp, Pagg, A_total):
    for i in 0 .. len(sieve)-2:
        fraction = (tpp[i] - tpp[i+1]) / (tpp[0] - tpp[-1])
        Aagg_i = fraction * Pagg * A_total
        store Aagg_i
    return Aagg

```

```

function consume_budget_with_remainder(interval_budget, remaining):
    target = interval_budget + remaining
    consumed = 0
    while target - consumed > epsilon:
        candidate = sample_particle_area()
        if consumed + candidate < target:
            consumed += candidate
            accept particle
        else:
            break
    return accepted_particles, target - consumed

```

C) Aggregate Generation per Worker

C1) Circular Worker (WorkerTodos)

```

function sample_circle_from_sieve_interval(d_min, d_max):
    d = uniform(d_min, d_max)
    r = d / 2
    area = pi * r^2
    return r, area

```

C2) Elliptical Worker (WorkerElipses)

```

function sample_ellipse_from_sieve_interval(d_min, d_max):
    d = uniform(d_min, d_max)
    b = d / 2
    aspect = uniform(1.0, 1.5)
    a = b * aspect
    area = pi * a * b
    return (a, b), area

```

```

function build_shapely_ellipse(x, y, a, b, angle):
    unit_circle = Point(x, y).buffer(1)
    ellipse = scale(unit_circle, a, b)
    ellipse = rotate(ellipse, angle, origin=(x, y))
    return ellipse

```

C3) Polygonal Worker (WorkerPoligonos)

```

function generate_irregular_polygon(target_area, max_tries=100):
    repeat max_tries:
        n_sides = random_int(5, 10)
        angles = equally_spaced(0, 2*pi, n_sides)
        radial_noise = uniform(0.8, 1.2, n_sides)

        vertices = polar_to_cartesian(radial_noise, angles)
        center vertices around origin

        current_area = polygon_area(vertices)
        if current_area == 0:
            continue

        scale_factor = sqrt(target_area / current_area)
        vertices = vertices * scale_factor

        if relative_area_error(vertices, target_area) < 0.05:
            polygon = Polygon(vertices)
            if invalid: polygon = polygon.buffer(0)
            return polygon

    return None

```

D) Aggregate Placement Core

```

function place_shape_list(shapes, domain, placed_aggregates, placed_pores,
    max_attempts=3000):
    for shape in shapes:
        placed = false
        for attempt in 1..max_attempts:
            candidate = random_translate_and_optional_rotate(shape)

            if not domain.contains(candidate):
                continue
            if intersects_any(candidate, placed_aggregates):
                continue
            if intersects_any(candidate, placed_pores):
                continue

            placed_aggregates.append(candidate)
            placed = true
            break

        if not placed:
            discard shape to avoid infinite loops in dense domains

```

E) Pore Radius Generation and Placement

```
function generate_pore_radii(A_total, Pporos, dmin, dmax):
    A_target = A_total * Pporos
    radii = []
    while A_target > pi * (dmin/2)^2:
        r = uniform(dmin/2, dmax/2)
        radii.append(r)
        A_target -= pi * r^2
    return radii
```

```
function place_pores(radii, domain, placed_aggregates, placed_pores):
    for r in radii:
        try random positions until accepted:
            fully inside domain
            no overlap with aggregates
            no overlap with previous pores
```

F) Reactive Point Radius Generation

```
function generate_reactive_radii(A_total, ratio, dmin, dmax):
    A_target = A_total * ratio
    radii = []
    while A_target > pi * (dmin/2)^2:
        r = uniform(dmin/2, dmax/2)
        radii.append(r)
        A_target -= pi * r^2
    return radii
```

G) Cluster-Based Reactive Point Placement

```
function generate_cluster_centers(domain, k_min, sigma):
    influence_radius = 3 * sigma
    cluster_area = pi * influence_radius^2
    k = max(k_min, ceil(domain.area / cluster_area))
    return [uniform_point_in_domain() for _ in 1..k]
```

```
function place_points_clustered(radii, centers, sigma, constraints):
    for r in radii:
        placed = false
        repeat up to max_attempts:
            (cx, cy) = random_choice(centers)
            x = normal(cx, sigma)
            y = normal(cy, sigma)

            if constraints_are_satisfied(x, y, r):
                save point
                placed = true
                break
        if not placed:
            discard radius
```

H) Simplex-Noise Reactive Point Placement

```
function place_points_simplex(radaii, simplex_seed=42, scale=0.05, threshold=0.3,
    constraints):
    noise = OpenSimplex(seed=simplex_seed)

    for r in radaii:
        placed = false
        repeat up to max_attempts:
            x, y = uniform_point_in_domain()
            v = (noise2(x * scale, y * scale) + 1) / 2

            if v < threshold:
                continue
            if constraints_are_satisfied(x, y, r):
                save point
                placed = true
                break
        if not placed:
            discard radius
```

I) Constraint Evaluation Logic

```
function constraints_are_satisfied(x, y, r):
    if not fully_inside_domain(x, y, r):
        return false
    if overlaps_existing_points(x, y, r):
        return false
    if overlaps_pores(x, y, r):
        return false

    if channel == "on_aggregates":
        if not intersects_at_least_one_aggregate(x, y, r):
            return false
    else if channel == "paste":
        if overlaps_forbidden_aggregate_region(x, y, r):
            return false

    return true
```

J) Practical Tuning Guide

```
If point distribution is too uniform:
    increase sigma in cluster mode
    reduce simplex threshold

If too many placement failures:
    reduce target ratios (Pagg, Pporos, reactive ratios)
    reduce min diameters
    increase max attempts carefully

If run is too slow:
    simplify geometry checks (circle mode)
```

lower sampling resolution for shape approximations
reduce plotting/export detail during development